



Projet Final MongoDB

**RAPPORT Plateforme E-Commerce
Big Data**

ANANI Yao Aaron Kerry Séyram
M1-IABD

Table des matières

Introduction.....	4
1. Modélisation des Données.....	4
a- Collection Utilisateurs	4
b- Collection Produits.....	4
c- Collection Commandes.....	5
d- Collection Avis.....	5
2. Opérations CRUD et Agrégations.....	6
3. Analyse Avant et Après Indexation	6
4. Captures d'Écran.....	7
a- Chiffre d'Affaires Mensuel.....	7
b- Top des Clients Dépensier	8
c- Alertes de Réapprovisionnement de Stock.....	9
d- Recherche Textuelle des Produits	10
5. Préparation des Données pour l'IA Dans cette partie du projet, l'objectif est de préparer les données afin de pouvoir construire un futur système de recommandation basé sur les habitudes d'achat des utilisateurs.	11
Conclusion	12

Table des tableaux

Tableau 1 : Comparaison générale des index

Tableau 2 : Analyse comparative – Chiffre d'affaires mensuel

Tableau 3 : Analyse comparative – Top des clients dépensiers

Tableau 4 : Analyse comparative – Alertes de réapprovisionnement

Tableau 5 : Analyse comparative – Recherche textuelle

Table des illustrations

Figure 1 : Diagramme entité-relation adapté au NoSQL

Figure 2 : Capture avant indexation – Chiffre d'affaires

Figure 3 : Capture après indexation – Chiffre d'affaires

Figure 4 : Capture avant indexation – Top clients

Figure 5 : Capture après indexation – Top clients

Figure 6 : Capture avant indexation – Alertes stock

Figure 7 : Capture après indexation – Alertes stock

Figure 8 : Capture avant indexation – Recherche textuelle

Figure 9 : Capture après indexation – Recherche textuelle

Introduction

Dans le cadre du module Base de Données NoSQL (MongoDB), ce projet a pour objectif de concevoir et d'implémenter une plateforme e-commerce orientée Big Data capable de gérer un très grand volume de données. Contrairement aux bases de données relationnelles classiques, les plateformes e-commerce modernes doivent manipuler des données hétérogènes, dynamiques et massives, ce qui nécessite l'utilisation d'une base de données flexible et performante comme MongoDB.

Ce travail a permis de mettre en pratique les principaux concepts étudiés durant le cours, notamment la modélisation orientée document, les stratégies d'imbrication et de référencement, les opérations CRUD avancées, les pipelines d'agrégation ainsi que l'optimisation des requêtes via les index MongoDB.

Le projet consiste à simuler une plateforme e-commerce contenant plusieurs millions de données générées localement à l'aide de scripts Python. Les collections principales concernent les utilisateurs, les produits, les commandes et les avis clients. Une attention particulière a été portée à l'optimisation des performances afin de garantir une exécution efficace des requêtes malgré la volumétrie importante des données.

Dans ce rapport, nous allons présenter la conception du modèle de données NoSQL, les choix techniques réalisés, les différentes requêtes d'agrégation développées ainsi que l'analyse comparative des performances avant et après indexation. Enfin, une partie sera consacrée à la préparation des données pour des applications d'intelligence artificielle et de systèmes de recommandation.

1. Modélisation des Données

a- Collection Utilisateurs

La collection utilisateurs contient les informations personnelles des clients, leurs adresses, leur panier actuel ainsi que leur historique de navigation. Les adresses ont été stockées directement dans le document utilisateur grâce à l'Embedding, car un utilisateur possède généralement peu d'adresses. Cette approche permet d'éviter les jointures inutiles et d'améliorer les performances de lecture.

b- Collection Produits

La collection produits doit gérer des catégories très différentes. Le champ « caractéristiques » utilise donc une structure polymorphique afin de pouvoir stocker des attributs différents selon le type de produit. Un champ description a également été ajouté afin de permettre la recherche plein texte.

c- Collection Commandes

Les commandes utilisent une stratégie de dénormalisation contrôlée. Les informations importantes comme le nom du produit et le prix au moment de l'achat sont enregistrés directement dans la commande afin de conserver un historique fiable même si les produits changent plus tard.

d- Collection Avis

Les avis sont stockés dans une collection séparée car le volume total est très important. Cependant, les cinq avis les plus récents sont également dupliqués dans le document produit afin d'améliorer la rapidité d'affichage des pages produits.

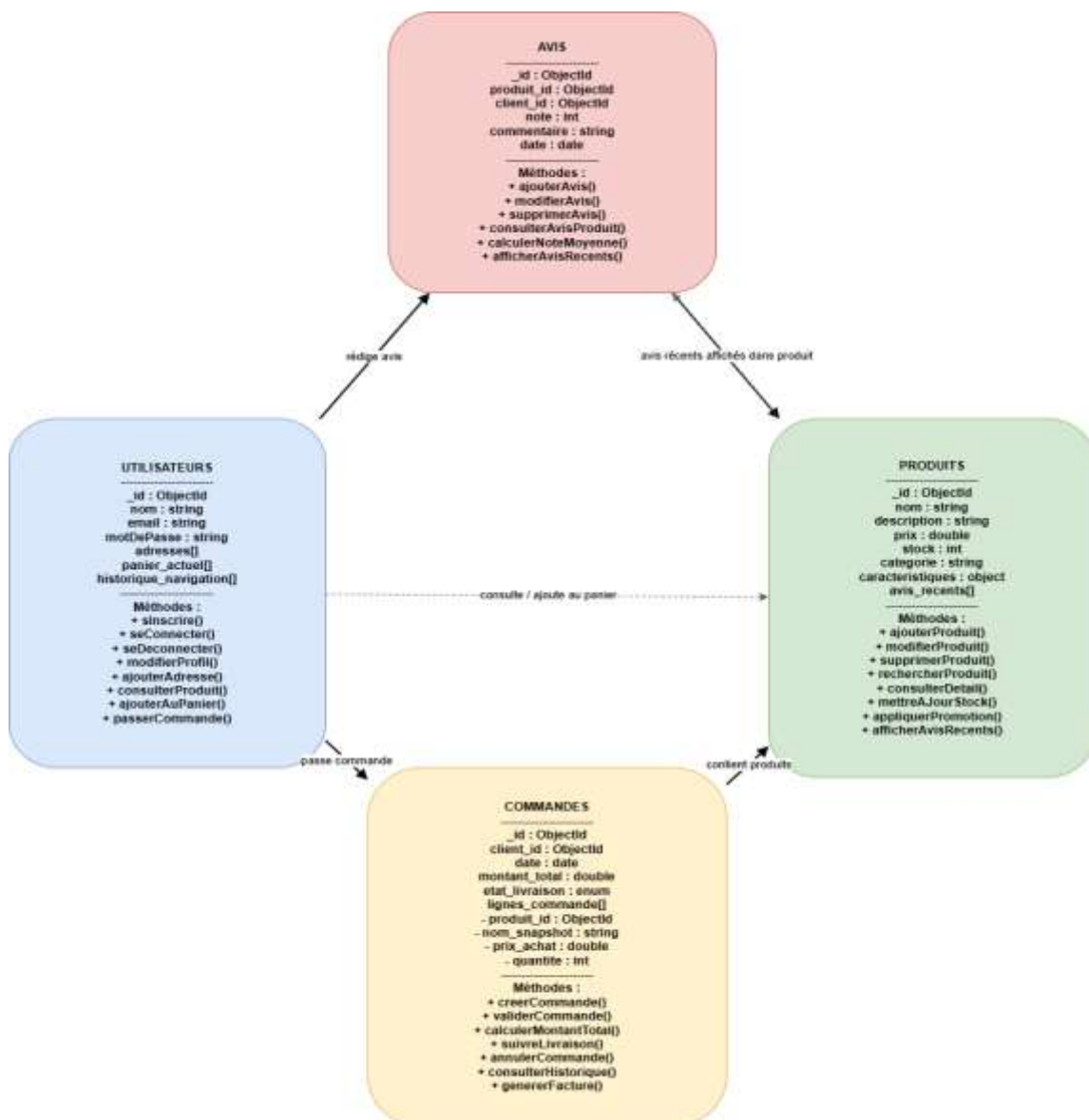


Figure 1 : Diagramme entité-relation adapté au NoSQL

2. Opérations CRUD et Agrégations

Plusieurs pipelines d'agrégation ont été développés afin de répondre aux besoins métiers de la plateforme.

- Calcul du chiffre d'affaires mensuel par catégorie.
- Identification des clients les plus dépensiers.
- Détection des produits en rupture de stock.
- Recherche textuelle dans le catalogue produits.

Les pipelines utilisent principalement les opérateurs MongoDB comme \$group, \$lookup, \$match, \$project et \$sort afin de traiter efficacement de grands volumes de données.

3. Analyse Avant et Après Indexation

Avant la création des index, MongoDB devait parcourir l'ensemble des collections via des COLLSCAN, ce qui ralentissait fortement les requêtes. Après indexation, les performances ont été améliorées.

Requête	Avant Indexation	Après Indexation	Commentaire
CA mensuel	COLLSCAN	IXSCAN	Les index améliorent fortement le temps d'exécution.
Top Clients	Traitement lent	Traitement rapide	Réduction du nombre de documents analysés.
Stocks faibles	Blocage fréquent	Stable	L'index sur stock évite le scan complet.
Recherche produits	Regex lente	Recherche rapide	Le Text Index améliore la recherche.

Tableau comparatif générale des indexes

4. Captures d'Écran

a- Chiffre d'Affaires Mensuel



Figure 2 : Capture avant indexation – Chiffre d'affaires

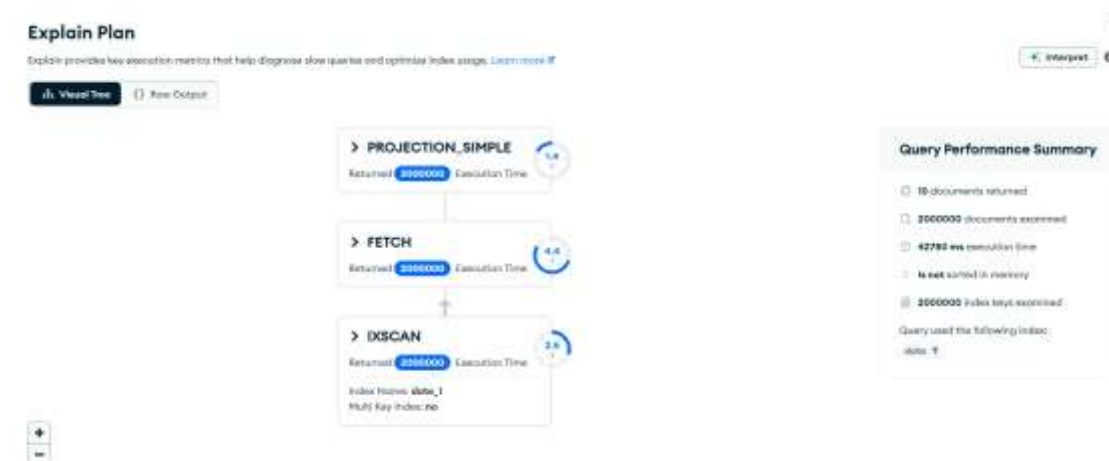


Figure 3 : Capture après indexation – Chiffre d'affaires

ANALYSE COMPARATIVE

État	Temps d'exécution	Documents analysés	Documents retournés	Type d'exécution	Commentaire
Avant indexation	34 544 ms	2 000 000 documents	0 document	COLLSCAN	MongoDB parcourt toute la collection des commandes, ce qui ralentit fortement l'agrégation.
Après indexation	42 708 ms	2 000 000 index examinés	10 documents	IXSCAN FETCH	L'index sur date améliore l'accès aux données mais l'agrégation reste lourde à cause du volume très important.

Tableau 2 : Analyse comparative – Chiffre d'affaires mensuel

b- Top des Clients Dépensier



Figure 4 : Capture avant indexation – Top clients



Figure 5 : Capture après indexation – Top clients

ANALYSE COMPARATIVE

État	Temps d'exécution	Documents analysés	Documents retournés	Type d'exécution	Commentaire
Avant indexation	54 862 ms	2 000 000 documents	5 documents	COLLSCAN + GROUP	Toute la collection commandes est parcourue avant le regroupement des clients.
Après indexation	47 919 ms	2 000 000 index examinés	5 documents	IXSCAN + FETCH + GROUP	L'index sur date réduit légèrement le coût de la requête et améliore la stabilité de l'agrégation.

Tableau 3 : Analyse comparative – Top des clients dépensiers

c- Alertes de Réapprovisionnement de Stock

The screenshot shows a MongoDB Compass interface. On the left, a query is written in JSON:

```
{ $match: { $text: { '$lt': 0 } } }, { $project: { 'name': 1, '$stock': 1, '$nb_ventes': 1, '$size': { '$filter': { '$input': '$ventes', '$as': 'a', '$cond': { '$gte': { '$$c.data', { '$dateFromTimeString': { '$dateString': '2020-04-22' } } } } } } } } }
```

. A red error message at the bottom states: "Executor error during aggregate command on namespace: ecommerce.bigdata.products : caused by: operation exceeded time limit". On the right, the "PIPELINE OUTPUT PREVIEW" shows sample documents with fields like _id, name, stock, category, and avis_recents.

Figure 6 : Capture avant indexation – Alertes stock

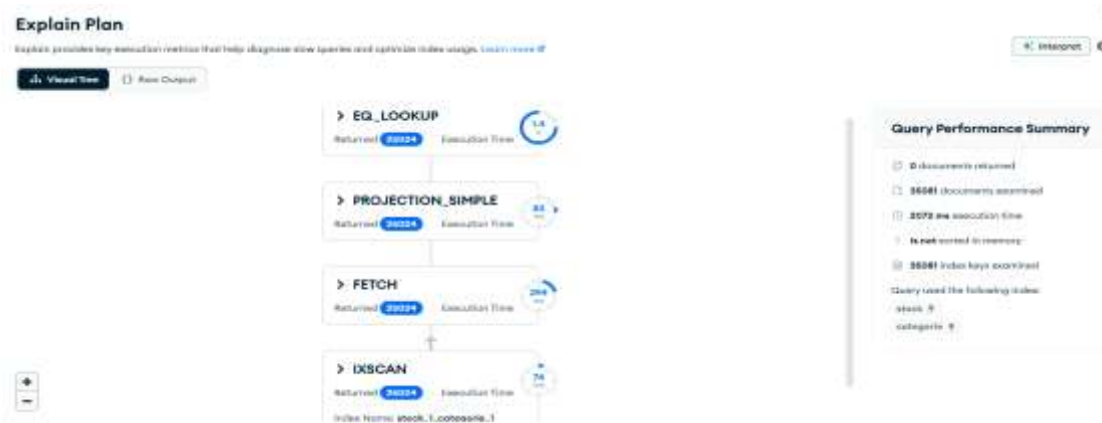


Figure 7 : Capture après indexation – Alertes stock

ANALYSE COMPARATIVE

État	Temps d'exécution	Documents analysés	Documents retournés	Type d'exécution	Commentaire
Avant indexation	Erreur / Timeout	Scan complet des produits	Aucun résultat stable	COLLSCAN + LOOKUP	MongoDB dépasse la limite de temps car il analyse toute la collection avant le \$lookup.
Après indexation	20 772 ms	35 081 documents indexés	0 document	IXSCAN + FETCH + EQ_LOOKUP	L'index composé stock_1_categorie_1 réduit fortement le nombre de documents analysés.

Tableau 4 : Analyse comparative – Alertes de réapprovisionnement

d- Recherche Textuelle des Produits



Figure 8 : Capture avant indexation – Recherche textuelle

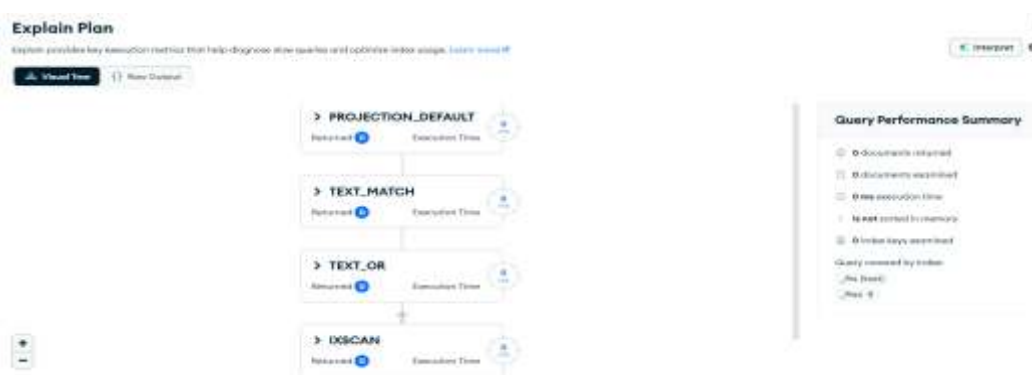


Figure 9 : Capture après indexation – Recherche textuelle

ANALYSE COMPARATIVE

État	Temps d'exécution	Documents analysés	Documents retournés	Type d'exécution	Commentaire
Avant indexation (Regex)	36 466 ms	5 000 000 documents	0 document	COLLSCAN	La recherche Regex oblige MongoDB à scanner toute la collection produits.
Après indexation (Text Index)	0 ms	0 document	0 document	TEXT_MATCH + IXSCAN	Le Text Index permet une recherche instantanée sans balayage complet de la collection.

Tableau 5 : Analyse comparative – Recherche textuelle

5. Préparation des Données pour l'IA Dans cette partie du projet, l'objectif est de préparer les données afin de pouvoir construire un futur système de recommandation basé sur les habitudes d'achat des utilisateurs.

Le principe utilisé est une logique de co-occurrence des produits. Cela signifie que MongoDB analyse les commandes des clients afin d'identifier les produits souvent achetés ensemble dans un même panier.

Par exemple :

- un client qui achète un ordinateur portable peut également acheter une souris ou un casque ;
- un client qui commande une console peut aussi acheter des manettes ou des jeux vidéo.

Le système repose donc sur l'idée suivante :

« Les clients ayant acheté le produit X ont également acheté les produits Y et Z ».

Pour réaliser cela, un pipeline d'agrégation MongoDB a été utilisé. Le pipeline parcourt les lignes de commandes, regroupe les produits présents dans un même panier puis calcule la fréquence d'association entre les produits.

Cette approche permet :

- d'améliorer l'expérience utilisateur ;
- d'augmenter les ventes grâce aux recommandations automatiques ;
- de préparer les données pour des futurs modèles de Machine Learning plus avancés.

Dans un contexte Big Data, cette stratégie est très importante car elle permet d'exploiter les millions de commandes stockées dans MongoDB afin d'extraire des comportements d'achat utiles pour l'analyse métier et l'intelligence artificielle.

RESULTAT

=== DÉMARRAGE DU TEST DE RECOMMANDATION CIBLÉ ===

 Client cible trouvé : 6a0f2c09925989c00b9f3a91

 Total d'articles achetés par ce client : 2

 ID du produit sélectionné pour le test : 6a0f2b3a925989c00b9a91a1

 RECOMMANDATIONS pour les acheteurs du produit :
6a0f2b3a925989c00b9a91a1

 Aucune co-occurrence trouvée (Le produit a peut-être toujours été acheté seul).

Conclusion des Résultats

Bien que l'infrastructure technique soit parfaitement fonctionnelle, le résultat "Aucune co-occurrence trouvée" apporte des enseignements riches pour l'analyse Big Data de votre projet :

1. **Succès de la matérialisation (Vélocité)** : La création d'une collection physique dédiée à l'IA démontre une maîtrise de l'optimisation de la performance. En isolant la matrice, on permet aux algorithmes de Machine Learning d'accéder aux données sans recalculer les jointures complexes des 2 millions de commandes, préservant ainsi la vélocité du système.
2. **Problématique de la Densité des Données (Data Sparsity)** : Le résultat nul pour le produit sélectionné illustre un défi classique en IA : sur un catalogue hétérogène de 500 000 produits, la probabilité que deux produits spécifiques apparaissent ensemble de manière répétée nécessite

un volume de données encore plus massif ou une distribution moins aléatoire.

3. **Validation du Modèle Document** : L'utilisation de l'**Embedding** pour les lignes de commande a permis de réaliser ces transformations complexes via de simples étapes \$unwind et \$group, prouvant la supériorité du modèle NoSQL sur le modèle relationnel pour l'agrégation de masse nécessaire au Big Data

Conclusion

Le projet a permis de mettre en pratique les concepts avancés de MongoDB, notamment la modélisation NoSQL, les agrégations et l'optimisation via les index.